Challenge 1: The banknote-authentication data set problem

1. Data Pretreatment

Data Preprocessing Report: Splitting and Scaling

To ensure a robust evaluation, I started by dividing the dataset using a **Stratified Train-Test Split**. This method was chosen to maintain the original proportion of classes (Genuine vs. Forged) in both subsets, preventing any representational bias during the training phase.
Following the split, I applied Data Scaling (e.g., **StandardScaler**) to normalize the feature range. Crucially, this scaling was performed after the split to prevent **Data Leakage**:
The scaler was fitted only on the **Training Set** to learn the parameters (mean and standard deviation).
These parameters were then used to transform the **Test Set**.
This strict separation ensures that the model has zero knowledge of the test data's statistical properties during training, guaranteeing that our final evaluation metrics reflect true performance on unseen data.

2. Unsupervised Learning

Analysis of PCA and K-Means Clustering

In this phase, I utilized **PCA** to reduce the dimensionality of the Banknote Authentication dataset and visualize the feature space. Subsequently, I applied the **K-Means algorithm** (with k=2) to both the PCA-reduced data and the full 4-dimensional dataset. The contingency analysis showed that **K-Means failed to separate the classes effectively**. The resulting clusters were highly mixed (approximately 50% purity in the worst cases), indicating that **the algorithm's assumption of spherical, equidistant clusters does not align with the actual geometric distribution of this dataset.**

Balancing Sensitivity and Precision in DBSCAN

In my unsupervised analysis, after tuning the `eps` parameter, finding the optimum as `eps=0.4`, and successfully locating several clusters, I explored the trade-off between noise reduction and cluster purity by tuning the `min_samples` hyperparameter. I conducted three targeted experiments. In the end the optimal trade-off was found with `min_samples=5.`It significantly reduced noise compared to the first attempt (down to 57 points) while avoiding the over-merging issues of the second. Interestingly, a small group of 8 genuine banknotes consistently appears within the main 'forged' cluster across different settings. This persistence suggests a genuine overlap in the feature space—these specific genuine notes are mathematically indistinguishable from forged ones based on density alone, rather than being an artifact of parameter selection.

Comparative Analysis: K-Means vs. DBSCAN

Comparing the two unsupervised learning techniques reveals a clear distinction in performance driven by the data's geometry:

- **K-Means** (The Failure): Due to its tendency to force data into spherical shapes, K-Means could not distinguish the intricate, non-linear boundaries between genuine and forged banknotes. It effectively 'cut' the dataset in half arbitrarily, resulting in clusters with no predictive value.

- **DBSCAN** (The Success): By relying on density rather than distance from a centroid, DBSCAN successfully identified the irregular, manifold structures of the data. Although it required parameter tuning (`eps`/`min_samples`) to manage noise, it succeeded in isolating high-purity sub-clusters of genuine and forged notes that K-Means completely missed.

Conclusion: For this specific domain, a density-based approach (DBSCAN) is vastly superior to a centroid-based approach (K-Means), as the underlying features form complex, continuous shapes rather than compact blobs.

3. Supervised Learning

Methodology Report: Hyperparameter Optimization

In this supervised learning phase, my goal was not just to train models, but to maximize their potential performance. To achieve this, I moved beyond default settings and implemented a systematic optimization strategy using **Grid Search with Cross-Validation (`GridSearchCV`)**.

For each algorithm, I defined a specific dictionary of hyperparameters to explore—such as the number of neighbors (**k**) for KNN, or the regularization strength (**C**) for Logistic Regression.

Once the Grid Search identified the 'Best Estimator' (the optimal combination of parameters), I automatically used this optimized model to make predictions on the held-out Test Set.

Comparative analysis

1. K-Nearest Neighbors (KNN) the winner

- **Parameters:** `n_neighbors=1`, `metric='euclidean'`
- **Accuracy:** 99.73% (0.9973)
- **Precision:** 99.40%
- **Recall:** 100.00%
- **F1-Score:** 99.70%

I found that 1-NN is the best performing model here, achieving nearly perfect accuracy.
I know that KNN is an instance-based learner that looks at the closest point in the training data. Since my earlier DBSCAN analysis showed that the classes form very dense, distinct (though non-spherical) clusters, I can see why a new point would almost always be extremely close to another point of the same class.
While I recognize that **k=1** is perfect for this dataset, I am aware it can sometimes overfit if there is noise (outliers). However, as I observed with DBSCAN, the "noise" in this data was mostly just points in lower density areas rather than mislabeled points deep inside the wrong cluster.

2. The others

Logistic Regression:	Decision Tree (ID3)	Gaussian Naive Bayes
Accuracy: 99.19% (0.9919) **Precision:** 100.00% **Recall:** 98.18% **F1-Score:** 99.08%	**Accuracy:** 98.12% (0.9812) **Precision:** 97.59% **Recall:** 98.18% **F1-Score:** 97.89%	**Accuracy:** 86.02% (0.8602) **Precision:** 85.99% **Recall:** 81.82% **F1-Score:** 83.85%

Even though I saw K-Means fail (likely due to using linear distances from centroids), Logistic Regression works by finding a linear decision boundary.
This high performance suggests to me that while the clusters might be shaped weirdly, they are likely linearly separable in the 4-dimensional feature space (or very close to it).

Decision Tree performed well, though slightly worse than KNN and Logistic Regression.
Since Decision Trees split data using horizontal and vertical lines (rectangles), I suspect that if the boundary between True and False notes is diagonal or curved, the tree has to create a very complex "staircase" boundary to separate them, leading to these small errors.

Naive Bayes is thhe worst performing model by a significant margin.
Reflecting on my DBSCAN analysis, I saw the clusters had weird, density-based shapes rather than nice round blobs. This violates the Gaussian assumption. I also know that features like kurtosis and skewness are related.

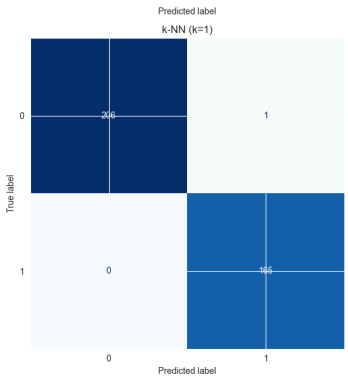
Comparative Analysis of Confusion Matrices and ROC Curves

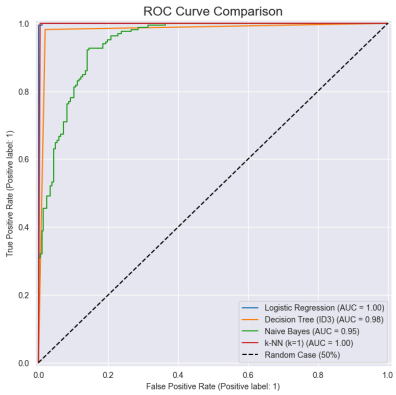
Visualizing the confusion matrices confirms the hypothesis derived from the accuracy scores: the geometry of the data dictates the success of the model. KNN and Logistic Regression have an **(AUC) of 1.00**, both models demonstrate perfect separability. However, when analysing the confusion matrices in detail, we notice some important differences.

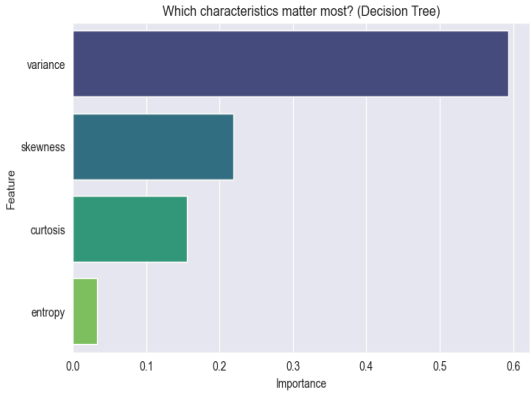
K-Nearest Neighbors made **only 1 single error.** It committed 1 False Positive, this model flagged one genuine note as a counterfeit. It let **zero** fake notes pass through. This is the ideal model for a bank. It is incredibly secure and virtually error-free, proving that the local density of points is the strongest predictor.

Logistic Regression made only **3 errors**. All 3 were False Negatives. This is slightly dangerous. It classified 3 fake notes as 'Real'. In a banking context, accepting counterfeit money is a worse error than rejecting a real note. The fact that its few errors were 'security breaches' makes it slightly inferior to KNN for this specific use case.

Decision Tree struggled slightly more than the previous two because it tries to draw boxy, rectangular boundaries. Since the data likely has diagonal or curved separation lines, the tree made small mistakes at the jagged edges of its decision regions. To understand the decision-making process of the ID3 Decision Tree, I extracted and visualized the 'Feature Importance' scores, finding that variance is the most decisive indicator of authenticity.







This confirms what we saw with PCA, where we saw that 95% of the information was explained by 3 components.

Decision Boundaries

To visualize the 'reasoning' behind each model's predictions, I projected the data into a 2D space using PCA and plotted the decision boundaries. This graphic serves as the visual confirmation of our numerical results.

1. **The Flexible Geography of KNN (Top-Left)**

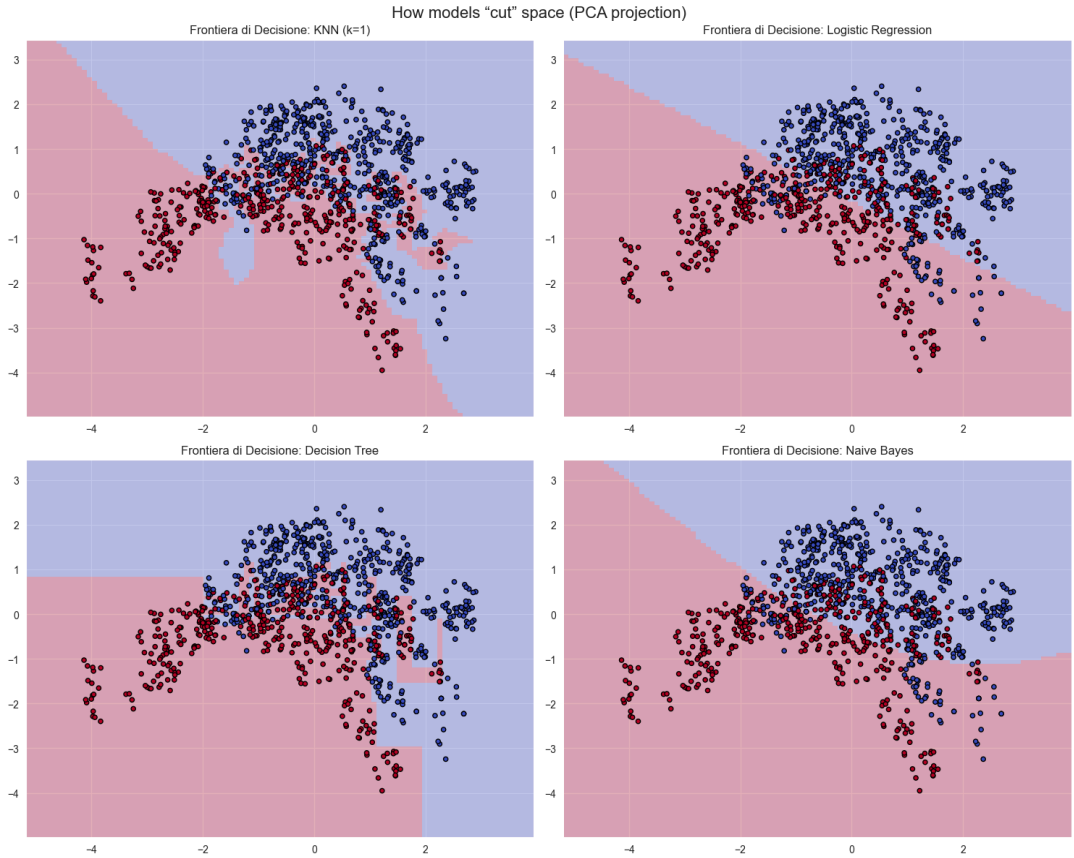
The KNN plot reveals a complex, fragmented decision landscape. Instead of forcing a geometric shape, the model adapts perfectly to the local density of the data, creating 'islands' and jagged coastlines. This flexibility allows it to capture the non-linear nuances of the dataset, explaining its superior performance.

2. **The Linear Cut of Logistic Regression (Top-Right)**

The Logistic Regression plot displays a stark, straight line dividing the space. The fact that this simple linear cut results in such high accuracy (99%+) is a major discovery: it proves that despite the complex internal structure of the clusters, the two main classes (Genuine vs. Forged) are linearly separable.

3. **The 'Staircase' of the Decision Tree (Bottom-Left)**

This plot clearly shows the orthogonal nature of the Decision Tree. Notice the stepped, blocky boundaries. The model attempts to approximate the diagonal separation line by creating a series of vertical and horizontal splits, effectively building a 'staircase' to separate the classes.

4. **The Mismatch of Naive Bayes (Bottom-Right)**

This plot visually explains the model's failure. The boundaries are smooth, elliptical curves—the result of the Gaussian assumption. It is painfully obvious here that the data does not fit into these smooth blobs. The rigid, rounded boundaries fail to capture the elongated and irregular shapes of the actual clusters, leading to the high error rate observed in the Confusion Matrix.